

The ETag or entity tag is part of HTTP, the protocol for the World Wide Web. It is one of several mechanisms that HTTP provides for Web cache validation, which allows a client to make conditional requests. This mechanism allows caches to be more efficient and saves bandwidth, as a Web server does not need to send a full response if the content has not changed. ETags can also be used for optimistic concurrency control to help prevent simultaneous updates of a resource from overwriting each other.

An ETag is an opaque identifier assigned by a Web server to a specific version of a resource found at a URL. If the resource representation at that URL ever changes, a new and different ETag is assigned. Used in this manner, ETags are similar to fingerprints and can quickly be compared to determine whether two representations of a resource are the same.

ETag generation

The use of ETags in the HTTP header is optional (not mandatory as with some other fields of the HTTP 1.1 header). The method by which ETags are generated has never been specified in the HTTP specification.

Common methods of ETag generation include using a collision-resistant hash function of the resource's content, a hash of the last modification timestamp, or even just a revision number.

In order to avoid the use of stale cache data, methods used to generate ETags should guarantee (as much as is practical) that each ETag is unique. However, an ETag-generation function could be judged to be "usable", if it can be proven (mathematically) that duplication of ETags would be "acceptably rare", even if it could or would occur.

RFC-7232 explicitly states that ETags should be content-coding aware, e.g.

ETag: "123-a" – for no Content-Encoding

ETag: "123-b" – for Content-Encoding: gzip

Some earlier checksum functions that were weaker than CRC32 or CRC64 are known to suffer from hash collision problems. Thus they were not good candidates for use in ETag generation.

Strong and weak validation

The ETag mechanism supports both strong validation and weak validation. They are distinguished by the presence of an initial "W/" in the ETag identifier, as:

"123456789" – A strong ETag validator

W/"123456789" – A weak ETag validator

A strongly validating ETag match indicates that the content of the two resource representations is byte-for-byte identical and that all other entity fields (such as Content-Language) are also unchanged. Strong ETags permit the caching and reassembly of partial responses, as with byte-range requests.

A weakly validating ETag match only indicates that the two representations are semantically equivalent, meaning that for practical purposes they are interchangeable and that cached copies can be used. However, the resource representations are not necessarily byte-for-byte identical, and thus weak ETags are not suitable for byte-range requests. Weak ETags may be useful for cases in which strong ETags are impractical for a Web server to generate, such as with dynamically generated content.

Typical usage

In typical usage, when a URL is retrieved, the Web server will return the resource's current representation along with its corresponding ETag value, which is placed in an HTTP response header "ETag" field:

ETag: "686897696a7c876b7e"

The client may then decide to cache the representation, along with its ETag. Later, if the client wants to retrieve the same URL resource again, it will first determine whether the locally cached version of the URL has expired (through the Cache-Control and the Expire headers). If the URL has not expired, it will retrieve the locally cached resource. If it is determined that the URL has expired (is stale), the client will send a request to the server that includes its previously saved copy of the ETag in the "If-None-Match" field.

If-None-Match: "686897696a7c876b7e"

On this subsequent request, the server may now compare the client's ETag with the ETag for the current version of the resource. If the ETag values match, meaning that the resource has not changed, the server may send back a very short response with a HTTP 304 Not Modified status. The 304 status tells the client that its cached version is still good and that it should use that.

However, if the ETag values do not match, meaning the resource has likely changed, a full response including the resource's content is returned, just as if ETags were not being used. In this case, the client may decide to replace its previously cached version with the newly returned representation of the resource and the new ETag.

ETag values can be used in Web page monitoring systems. Efficient Web page monitoring is hindered by the fact that most websites do not set the ETag headers for Web pages. When a Web monitor has no hints whether Web content has been changed, all content has to be retrieved and analyzed using computing resources for both the publisher and subscriber.

Mismatched ETag detection

A buggy website can at times fail to update the ETag after its semantic resource has been updated. As of 2019, an example of a prominent such site is export.arxiv.org. As a result, the incorrectly returned response is status 304, and the client fails to retrieve the updated resource. To detect such a buggy website:

Cache the response and ETag, assuming there is an ETag and that the response was not aborted.

For a subsequent request that would've included the If-None-Match header, do not send this header with perhaps a random 20% probability. With this probability, if the response returns an altered content but the same ETag as what was previously cached, mark the website as buggy and disable ETag caching for it. As a reminder, for a strong ETag, the content comparison can be byte-for-byte, whereas, for a weak ETag, it would check semantic equivalence only.

Tracking using ETags

ETags can be used to track unique users, as HTTP cookies are increasingly being deleted by privacy-aware users. In July 2011, Ashkan Soltani and a team of researchers at UC Berkeley reported that a number of websites, including Hulu, were using ETags for tracking purposes. Hulu and KISSmetrics have both ceased "respawning" as of 29 July 2011, as KISSmetrics and over 20 of its clients are facing a class-action lawsuit over the use of "undeletable" tracking cookies partially involving the use of ETags.

Because ETags are cached by the browser and returned with subsequent requests for the same resource, a tracking server can simply repeat any ETag received from the browser to ensure an assigned ETag persists indefinitely (in a similar way to persistent cookies). Additional caching headers can also enhance the preservation of ETag data.

ETags may be flushable by clearing the browser cache (implementations vary).

References

External links

[Apache HTTP Server Documentation – FileETag Directive](#)

[Editing the Web: Detecting the Lost Update Problem Using Unreserved Checkout](#), W3C Note, 10 May 1999.

[Live demo of zombie cookie using ETags](#)

[Old SQUID Development projects – ETag support Archived 23 September 2012 at the Wayback Machine](#) (completed in 2001)

[Using ETags to Reduce Bandwidth & Workload with Spring & Hibernate](#)

[ETag in HTTP/1.1 specification](#)

[Concerning Etags and Datestamps](#) by Lars R. Clausen (2004)